

Fatio, Rationally Reconstructed: A Faithful Deep and Shallow Embedding of a Multi-Agent Argumentation Protocol

Christoph Benz Müller Luca Pasetto

August 2, 2026

Abstract

We present a rational reconstruction of the LogiKEy implementation of the Fatio dialogue protocol for multi-agent dialectical argumentation (Pasetto and Benz Müller, ARQNL 2024), rebuilt following the methodology of “Faithful Logic Embeddings in HOL — Deep and Shallow” (Benz Müller 2025; AFP entry FaithfulPMLinHOL). The object logic of Fatio’s axiomatic semantics — a multi-agent belief/desire logic over a propositional content logic, enriched with Done-, entailment- and existential-justification atoms — is developed both as a deep embedding (datatype syntax with a primitive-recursive Kripke semantics over models with explicit world domains) and as two shallow embeddings — a *maximal* one (model-parametric truth-set functions) and a *minimal* one in the classical LogiKEy style (fixed meta-level constants, truth sets over worlds) — all connected by machine-checked faithfulness theorems, including a soundness characterisation relating the two shallow backends and a documented comparison of their respective merits. The protocol layer (dialectical obligation stores, pre- and post-conditions, recursive dialogue checking) is defined over the deep syntax, so that syntactically distinct but semantically equivalent contents are discriminated exactly where the protocol requires it; the discrimination and identification facts, as well as the quantifier-scope question raised in the ARQNL paper, become theorems of the reconstruction.

Contents

1 Preliminaries	2
2 Deep embedding: content logic, Fatio language, BD logic	3
2.1 Content logic (deeply embedded, propositional)	3
2.2 The Fatio language	3
2.3 BD logic (deeply embedded)	3
2.4 Kripke semantics over models $\langle W, B, D, V, U, E \rangle$	4

3	Shallow embedding of the BD logic (maximal)	4
4	Shallow embedding of the BD logic (minimal)	5
5	Faithfulness	6
5.1	Discrimination vs. identification: the Done-problem, resolved	6
5.2	The quantifier-scope question, settled	7
5.3	Faithfulness for the minimal embedding, and the comparison	8
6	The Fatio protocol over the deep syntax	9
6.1	Dialectical obligation stores	9
6.2	Pre-conditions (axiomatic semantics)	10
6.3	Post-conditions and state evolution	10
6.4	Sanity theorems	11
7	Tests: two dialogues, checked with high automation	11
7.1	The full Brigade Restaurant dialogue (from the literature) . .	11
7.2	A created example: role reversal with a negative retraction .	13

1 Preliminaries

```
theory FatioFaithful-preliminaries
  imports Main
begin
```

```
typedecl w — type of possible worlds
typedecl  $\mathcal{S}$  — type of propositional constant symbols of the content logic
consts  $p::\mathcal{S}$   $q::\mathcal{S}$   $r::\mathcal{S}$   $n::\mathcal{S}$   $m::\mathcal{S}$   $s::\mathcal{S}$  — some content atoms (as in the Fatio paper)
datatype Speaker =  $a \mid b \mid c$  — the dialogue participants
```

```
type-synonym  $\mathcal{W} = w \Rightarrow bool$  — world domains
type-synonym  $\mathcal{R} = \textit{Speaker} \Rightarrow w \Rightarrow w \Rightarrow bool$  — agent-indexed accessibility relations
type-synonym  $\mathcal{V} = \mathcal{S} \Rightarrow w \Rightarrow bool$  — valuations of content atoms
```

— Bounded universal quantifier, as in [2]

```
abbreviation(input) BoundedAll:: $\mathcal{W} \Rightarrow \mathcal{W} \Rightarrow bool$  where BoundedAll  $W$   $\varphi \equiv \forall x.$ 
 $W\ x \longrightarrow \varphi\ x$ 
```

```
syntax -BoundedAll::  $pttrn \Rightarrow \mathcal{W} \Rightarrow bool \Rightarrow bool$   $((\exists \forall (-/:-)/ -) [0, 0, 10] 10)$ 
```

```
translations  $\forall x.W. \varphi \rightleftharpoons \textit{CONST BoundedAll } W (\lambda x. \varphi)$ 
```

```
declare[[syntax-ambiguity-warning=false]]
```

```
end
```

2 Deep embedding: content logic, Fatio language, BD logic

```
theory FatioFaithful-deep
  imports FatioFaithful-preliminaries
begin
```

2.1 Content logic (deeply embedded, propositional)

```
datatype Formula = AtmC  $\mathcal{S}$  ( $\neg^c$  [1000] 999) | NegC Formula ( $\neg^c$ - [96] 96)
  | ImpC Formula Formula (infixr  $\supset^c$  93)
definition AndC (infixr  $\wedge^c$  95) where  $\varphi \wedge^c \psi \equiv \neg^c(\varphi \supset^c \neg^c \psi)$ 
definition OrC (infixr  $\vee^c$  92) where  $\varphi \vee^c \psi \equiv \neg^c \varphi \supset^c \psi$ 
definition TopC ( $\top^c$ ) where  $\top^c \equiv p^c \supset^c p^c$ 
definition BotC ( $\perp^c$ ) where  $\perp^c \equiv \neg^c \top^c$ 
```

2.2 The Fatio language

```
datatype Sign = Pls ( $\oplus$ ) | Mns ( $\ominus$ )
datatype FatioL =
  Assert Speaker Formula (assert[-,-])
  | Question Speaker Speaker Formula (question[-,-,-])
  | Challenge Speaker Speaker Formula (challenge[-,-,-])
  | Justify Speaker Formula Sign Formula (justify[-,-,-])
  | Retract Speaker Formula Sign (retract[-,-,-])
```

2.3 BD logic (deeply embedded)

The object language of the axiomatic semantics of Fatio [6], as formalised in [7]: agent beliefs $\mathcal{B}^d$ and desires $\mathcal{D}^d$ over the content logic, with three further kinds of atoms: $Done^d$ for uttered locutions, $EntD$ for argumentative entailment, and $ExJD$ i φ whose single semantic clause carries the existential of the pre-conditions of question and challenge, $(\exists \Delta) Done[justify(i, \Delta \vdash^{sg} \varphi)]$, inside the object language. Following the sign-parametric justify locution of the LogiKEy sources [3], $ExJD$ carries the sign of the requested justification.

```
datatype BDF =
  AtmD  $\mathcal{S}$  ( $\neg^a$  [1000] 999)
  | DoneD FatioL ( $Done^d$ )
  | EntD Formula Sign Formula
  | ExJD Speaker Sign Formula
  | NegD BDF ( $\neg^d$ - [96] 96)
  | ImpD BDF BDF (infixr  $\supset^d$  93)
  | BelD Speaker BDF ( $\mathcal{B}^d$ )
  | DesD Speaker BDF ( $\mathcal{D}^d$ )
```

```
definition AndD (infixr  $\wedge^d$  95) where  $\varphi \wedge^d \psi \equiv \neg^d(\varphi \supset^d \neg^d \psi)$ 
definition OrD (infixr  $\vee^d$  92) where  $\varphi \vee^d \psi \equiv \neg^d \varphi \supset^d \psi$ 
```

— Homomorphic injection of content formulas into the BD language

primrec *MapC* :: *Formula* $\Rightarrow$ *BDF* ($\{-\}^d$) **where**
 $\{\varphi^c\}^d = \varphi^a \mid \{\neg^c \varphi\}^d = \neg^d \{\varphi\}^d \mid \{\varphi \supset^c \psi\}^d = \{\varphi\}^d \supset^d \{\psi\}^d$

2.4 Kripke semantics over models $\langle W, B, D, V, U, E \rangle$

type-synonym $\mathcal{U} = \text{FatioL} \Rightarrow w \Rightarrow \text{bool}$ — Done-valuations

type-synonym $\mathcal{E} = \text{Formula} \Rightarrow \text{Sign} \Rightarrow \text{Formula} \Rightarrow w \Rightarrow \text{bool}$ — entailment-valuations

primrec *RelTD* :: $\mathcal{W} \Rightarrow \mathcal{R} \Rightarrow \mathcal{R} \Rightarrow \mathcal{V} \Rightarrow \mathcal{U} \Rightarrow \mathcal{E} \Rightarrow w \Rightarrow \text{BDF} \Rightarrow \text{bool}$ ($\langle -, -, -, -, -, - \rangle, - \models^d -$) **where**
 $\langle W, B, D, V, U, E \rangle, w \models^d x^a = V x w$
 $\mid \langle W, B, D, V, U, E \rangle, w \models^d \text{Done}^d l = U l w$
 $\mid \langle W, B, D, V, U, E \rangle, w \models^d \text{EntD } \Phi \text{ sg } \varphi = E \Phi \text{ sg } \varphi w$
 $\mid \langle W, B, D, V, U, E \rangle, w \models^d \text{ExJD } i \text{ sg } \varphi = (\exists \Delta. U (\text{Justify } i \Delta \text{ sg } \varphi) w)$
 $\mid \langle W, B, D, V, U, E \rangle, w \models^d \neg^d \varphi = (\neg \langle W, B, D, V, U, E \rangle, w \models^d \varphi)$
 $\mid \langle W, B, D, V, U, E \rangle, w \models^d \varphi \supset^d \psi = (\langle W, B, D, V, U, E \rangle, w \models^d \varphi \longrightarrow \langle W, B, D, V, U, E \rangle, w \models^d \psi)$
 $\mid \langle W, B, D, V, U, E \rangle, w \models^d \mathcal{B}^d i \varphi = (\forall v: W. B i w v \longrightarrow \langle W, B, D, V, U, E \rangle, v \models^d \varphi)$
 $\mid \langle W, B, D, V, U, E \rangle, w \models^d \mathcal{D}^d i \varphi = (\forall v: W. D i w v \longrightarrow \langle W, B, D, V, U, E \rangle, v \models^d \varphi)$

definition *ValD* ($\models^d -$) **where** $\models^d \varphi \equiv \forall W B D V U E. \forall w: W. \langle W, B, D, V, U, E \rangle, w \models^d \varphi$

— Global consequence from a set of deep premises, over all models

definition *ConsD* :: $(\text{BDF} \Rightarrow \text{bool}) \Rightarrow \text{BDF} \Rightarrow \text{bool}$ (**infix** $\models$ 25) **where**
 $\Gamma \models \varphi \equiv \forall W B D V U E. (\forall \gamma. \Gamma \gamma \longrightarrow (\forall w: W. \langle W, B, D, V, U, E \rangle, w \models^d \gamma))$
 $\longrightarrow (\forall w: W. \langle W, B, D, V, U, E \rangle, w \models^d \varphi)$

named-theorems *DefD*

declare *AndC-def*[*DefD, simp*] *OrC-def*[*DefD, simp*] *TopC-def*[*DefD, simp*] *BotC-def*[*DefD, simp*]
AndD-def[*DefD, simp*] *OrD-def*[*DefD, simp*] *ValD-def*[*DefD*] *ConsD-def*[*DefD*]

end

3 Shallow embedding of the BD logic (maximal)

theory *FatioFaithful-shallow*

imports *FatioFaithful-deep*

begin

type-synonym $\sigma = \mathcal{W} \Rightarrow \mathcal{R} \Rightarrow \mathcal{R} \Rightarrow \mathcal{V} \Rightarrow \mathcal{U} \Rightarrow \mathcal{E} \Rightarrow w \Rightarrow \text{bool}$

definition *AtmS* :: $\mathcal{S} \Rightarrow \sigma$ ($-^s$ [1000] 999) **where** $x^s \equiv \lambda W B D V U E w. V x w$

definition *DoneS* :: *FatioL* $\Rightarrow \sigma$ (*Done*^s) **where** $\text{Done}^s l \equiv \lambda W B D V U E w. U l w$

definition *EntS* :: *Formula* $\Rightarrow \text{Sign} \Rightarrow \text{Formula} \Rightarrow \sigma$ **where** $\text{EntS } \Phi \text{ sg } \varphi \equiv \lambda W B D V U E w. E \Phi \text{ sg } \varphi w$

definition $ExJS::Speaker \Rightarrow Sign \Rightarrow Formula \Rightarrow \sigma$ **where**
 $ExJS\ i\ sg\ \varphi \equiv \lambda W\ B\ D\ V\ U\ E\ w. \exists \Delta. U\ (Justify\ i\ \Delta\ sg\ \varphi)\ w$
definition $NegS::\sigma \Rightarrow \sigma$ ($\neg^s$ - [96] 96) **where** $\neg^s \varphi \equiv \lambda W\ B\ D\ V\ U\ E\ w. \neg\ \varphi\ W\ B\ D\ V\ U\ E\ w$
definition $ImpS::\sigma \Rightarrow \sigma \Rightarrow \sigma$ (**infixr** $\supset^s$ 93) **where**
 $\varphi \supset^s \psi \equiv \lambda W\ B\ D\ V\ U\ E\ w. \varphi\ W\ B\ D\ V\ U\ E\ w \longrightarrow \psi\ W\ B\ D\ V\ U\ E\ w$
definition $BelS::Speaker \Rightarrow \sigma \Rightarrow \sigma$ ($\mathcal{B}^s$) **where**
 $\mathcal{B}^s\ i\ \varphi \equiv \lambda W\ B\ D\ V\ U\ E\ w. \forall v:W. B\ i\ w\ v \longrightarrow \varphi\ W\ B\ D\ V\ U\ E\ v$
definition $DesS::Speaker \Rightarrow \sigma \Rightarrow \sigma$ ($\mathcal{D}^s$) **where**
 $\mathcal{D}^s\ i\ \varphi \equiv \lambda W\ B\ D\ V\ U\ E\ w. \forall v:W. D\ i\ w\ v \longrightarrow \varphi\ W\ B\ D\ V\ U\ E\ v$
definition $RelTS::\mathcal{W} \Rightarrow \mathcal{R} \Rightarrow \mathcal{R} \Rightarrow \mathcal{V} \Rightarrow \mathcal{U} \Rightarrow \mathcal{E} \Rightarrow w \Rightarrow \sigma \Rightarrow bool$ ($\langle -, -, -, -, -, - \rangle, - \models^s -$) **where**
 $\langle W, B, D, V, U, E \rangle, w \models^s \varphi \equiv \varphi\ W\ B\ D\ V\ U\ E\ w$
definition $ValS (\models^s -)$ **where** $\models^s \varphi \equiv \forall W\ B\ D\ V\ U\ E. \forall w:W. \langle W, B, D, V, U, E \rangle, w \models^s \varphi$

named-theorems $DefS$

declare $AtmS-def[DefS, simp]$ $DoneS-def[DefS, simp]$ $EntS-def[DefS, simp]$ $ExJS-def[DefS, simp]$
 $NegS-def[DefS, simp]$ $ImpS-def[DefS, simp]$ $BelS-def[DefS, simp]$ $DesS-def[DefS, simp]$
 $RelTS-def[DefS, simp]$ $ValS-def[DefS]$

end

4 Shallow embedding of the BD logic (minimal)

theory *FatIoFaithful-shallow-minimal*

imports *FatIoFaithful-deep*

begin

The minimal shallow embedding fixes one model at the meta level: the accessibility relations and valuations are uninterpreted HOL constants, formulas are plain truth sets over worlds, and the world domain is implicitly the full type w . This is the classical SSE style of LogiKEy [3] and the style of the original HOMML/FATIO development; the price, made explicit by the faithfulness theorems below, is that faithfulness holds relative to full-domain models only.

consts $B_0::\mathcal{R}$ $D_0::\mathcal{R}$ $V_0::\mathcal{V}$ $U_0::\mathcal{U}$ $E_0::\mathcal{E}$

consts $V_1::\mathcal{V}$ — a second valuation, used for a countermodel below

type-synonym $\sigma_m = w \Rightarrow bool$

definition $AtmM::S \Rightarrow \sigma_m$ ($-^m$ [1000] 999) **where** $x^m \equiv \lambda w. V_0\ x\ w$

definition $DoneM::FatIoL \Rightarrow \sigma_m$ ($Done^m$) **where** $Done^m\ l \equiv \lambda w. U_0\ l\ w$

definition $EntM::Formula \Rightarrow Sign \Rightarrow Formula \Rightarrow \sigma_m$ **where** $EntM\ \Phi\ sg\ \varphi \equiv \lambda w. E_0\ \Phi\ sg\ \varphi\ w$

definition $ExJM::Speaker \Rightarrow Sign \Rightarrow Formula \Rightarrow \sigma_m$ **where**

$ExJM\ i\ sg\ \varphi \equiv \lambda w. \exists \Delta. U_0\ (Justify\ i\ \Delta\ sg\ \varphi)\ w$

definition $NegM::\sigma_m \Rightarrow \sigma_m$ ($-^m$ - [96] 96) **where** $\neg^m \varphi \equiv \lambda w. \neg\ \varphi\ w$

definition $ImpM::\sigma_m \Rightarrow \sigma_m \Rightarrow \sigma_m$ (**infixr** $\supset^m$ 93) **where** $\varphi \supset^m \psi \equiv \lambda w. \varphi \ w \longrightarrow \psi \ w$
definition $BelM::Speaker \Rightarrow \sigma_m \Rightarrow \sigma_m$ ($\mathcal{B}^m$) **where** $\mathcal{B}^m \ i \ \varphi \equiv \lambda w. \forall v. B_0 \ i \ w \ v \longrightarrow \varphi \ v$
definition $DesM::Speaker \Rightarrow \sigma_m \Rightarrow \sigma_m$ ($\mathcal{D}^m$) **where** $\mathcal{D}^m \ i \ \varphi \equiv \lambda w. \forall v. D_0 \ i \ w \ v \longrightarrow \varphi \ v$

definition $RelTM::w \Rightarrow \sigma_m \Rightarrow bool$ ($- \models^m -$) **where** $w \models^m \varphi \equiv \varphi \ w$
definition $ValM$ ($\models^m -$) **where** $\models^m \varphi \equiv \forall w. \varphi \ w$

named-theorems $DefM$

declare $AtmM-def[DefM,simp]$ $DoneM-def[DefM,simp]$ $EntM-def[DefM,simp]$ $ExJM-def[DefM,simp]$
 $NegM-def[DefM,simp]$ $ImpM-def[DefM,simp]$ $BelM-def[DefM,simp]$ $DesM-def[DefM,simp]$
 $RelTM-def[DefM,simp]$ $ValM-def[DefM]$

end

5 Faithfulness

theory *FatioFaithful-faithfulness*

imports *FatioFaithful-shallow FatioFaithful-shallow-minimal*
begin

— Mapping: deep to (maximal) shallow

primrec $DpToSh :: BDF \Rightarrow \sigma$ ($\lceil - \rceil$) **where**

$\lceil x^a \rceil = x^s \mid \lceil Done^d \ l \rceil = Done^s \ l \mid \lceil EntD \ \Phi \ sg \ \varphi \rceil = EntS \ \Phi \ sg \ \varphi$
 $\mid \lceil ExJD \ i \ sg \ \varphi \rceil = ExJS \ i \ sg \ \varphi \mid \lceil \neg^d \varphi \rceil = \neg^s \lceil \varphi \rceil \mid \lceil \varphi \supset^d \psi \rceil = \lceil \varphi \rceil \supset^s \lceil \psi \rceil$
 $\mid \lceil \mathcal{B}^d \ i \ \varphi \rceil = \mathcal{B}^s \ i \ \lceil \varphi \rceil \mid \lceil \mathcal{D}^d \ i \ \varphi \rceil = \mathcal{D}^s \ i \ \lceil \varphi \rceil$

— Automated faithfulness proofs, as in [1, 2]

theorem *Faithful1a*:

$\forall W \ B \ D \ V \ U \ E. \forall w:W. \langle W, B, D, V, U, E \rangle, w \models^d \varphi \longleftrightarrow \langle W, B, D, V, U, E \rangle, w \models^s \lceil \varphi \rceil$

apply (*induct* φ) **by** *auto*

theorem *Faithful1b*: $\models^d \varphi \longleftrightarrow \models^s \lceil \varphi \rceil$

using *Faithful1a* **unfolding** $ValD-def$ $ValS-def$ **by** *auto*

5.1 Discrimination vs. identification: the Done-problem, resolved

Syntactically, $p \supset^c p$ and $\neg^c \perp^c$ are different contents; hence locutions and Done-atoms over them are different objects. Semantically, their injections into the BD logic are equivalent. All four facts are theorems of the reconstruction.

lemma *ContentDiscriminated*: $(p^c \supset^c p^c) \neq \neg^c \perp^c$ **by** *simp*

lemma *LocutionDiscriminated*: $assert[a, p^c \supset^c p^c] \neq assert[a, \neg^c \perp^c]$ **by** *simp*

lemma *DoneDiscriminated*: $\text{Done}^d \text{ assert}[a, p^c \supset^c p^c] \neq \text{Done}^d \text{ assert}[a, \neg^c \perp^c]$
by *simp*
lemma *SemanticsIdentifies*:
 $\models^d (\{p^c \supset^c p^c\}^d \supset^d \{\neg^c \perp^c\}^d) \wedge \models^d (\{\neg^c \perp^c\}^d \supset^d \{p^c \supset^c p^c\}^d)$
unfolding *ValD-def* **by** *simp*

5.2 The quantifier-scope question, settled

The wide-scope (meta-level) existential of the pre-conditions of question and challenge implies the narrow-scope (object-level, via *ExJD*) reading.

lemma *ScopeImp*:

assumes $\exists \Delta. \Gamma \models \mathcal{D}^d j (\mathcal{B}^d k (\mathcal{D}^d j (\text{Done}^d (\text{Justify } i \Delta \text{ sg } \varphi))))$
shows $\Gamma \models \mathcal{D}^d j (\mathcal{B}^d k (\mathcal{D}^d j (\text{ExJD } i \text{ sg } \varphi)))$

proof –

obtain Δ **where** $H: \Gamma \models \mathcal{D}^d j (\mathcal{B}^d k (\mathcal{D}^d j (\text{Done}^d (\text{Justify } i \Delta \text{ sg } \varphi))))$ **using** *assms* **by** *blast*

show *?thesis* **unfolding** *ConsD-def*

proof (*intro allI impI*)

fix $W B D V U E w$

assume *sat*: $\forall \gamma. \Gamma \gamma \longrightarrow (\forall w: W. \langle W, B, D, V, U, E \rangle, w \models^d \gamma)$ **and** $wW: W w$
have $\langle W, B, D, V, U, E \rangle, w \models^d \mathcal{D}^d j (\mathcal{B}^d k (\mathcal{D}^d j (\text{Done}^d (\text{Justify } i \Delta \text{ sg } \varphi))))$

using $H[\text{unfolded } \text{ConsD-def}]$ *sat* wW **by** *blast*

thus $\langle W, B, D, V, U, E \rangle, w \models^d \mathcal{D}^d j (\mathcal{B}^d k (\mathcal{D}^d j (\text{ExJD } i \text{ sg } \varphi)))$ **by** *simp blast*

qed

qed

The converse fails: the narrow-scope reading does NOT imply the wide-scope one. This is where a model finder earns its keep - the statement is a non-theorem, and *nitpick* [5] refutes it in seconds, from which the countermodel below was read off. Two worlds suffice: at each of them a justification is uttered, but for DIFFERENT supports, so no single Δ works globally. We verify the countermodel explicitly, so that the entry stays independent of the model finder at build time.

lemma *ScopeConverse-countermodel*:

fixes $\varphi_1 \varphi_2 :: \text{Formula}$ **and** $w_1 w_2 :: w$ **and** $i::\text{Speaker}$ **and** $\text{sg}::\text{Sign}$ **and** $\varphi::\text{Formula}$

assumes *distinct* Δ : $\varphi_1 \neq \varphi_2$ **and** *distinct* w : $w_1 \neq w_2$

and $W: W = (\lambda w. w = w_1 \vee w = w_2)$

and $U: U = (\lambda l w. l = \text{Justify } i \text{ (if } w = w_1 \text{ then } \varphi_1 \text{ else } \varphi_2) \text{ sg } \varphi)$

shows $(\forall w: W. \langle W, B, D, V, U, E \rangle, w \models^d \text{ExJD } i \text{ sg } \varphi)$

$\wedge \neg(\exists \Delta. \forall w: W. \langle W, B, D, V, U, E \rangle, w \models^d \text{Done}^d (\text{Justify } i \Delta \text{ sg } \varphi))$

proof

show $\forall w: W. \langle W, B, D, V, U, E \rangle, w \models^d \text{ExJD } i \text{ sg } \varphi$ **using** U **by** *auto*

next

show $\neg(\exists \Delta. \forall w: W. \langle W, B, D, V, U, E \rangle, w \models^d \text{Done}^d (\text{Justify } i \Delta \text{ sg } \varphi))$

proof

assume $\exists \Delta. \forall w: W. \langle W, B, D, V, U, E \rangle, w \models^d \text{Done}^d (\text{Justify } i \Delta \text{ sg } \varphi)$

then obtain Δ **where** $U (\text{Justify } i \Delta \text{ sg } \varphi) w_1$ **and** $U (\text{Justify } i \Delta \text{ sg } \varphi) w_2$

using W by *auto*
 hence $\Delta = \varphi_1$ and $\Delta = \varphi_2$ using U *distinctw* by *auto*
 thus *False* using *distinct* Δ by *simp*
 qed
 qed

5.3 Faithfulness for the minimal embedding, and the comparison

primrec $DpToShMin :: BDF \Rightarrow \sigma_m$ ($\lceil \cdot \rceil$) **where**
 $\lceil x^a \rceil = x^m \mid \lceil Done^d l \rceil = Done^m l \mid \lceil EntD \Phi sg \varphi \rceil = EntM \Phi sg \varphi$
 $\mid \lceil ExJD i sg \varphi \rceil = ExJM i sg \varphi \mid \lceil \neg^d \varphi \rceil = \neg^m \lceil \varphi \rceil \mid \lceil \varphi \supset^d \psi \rceil = \lceil \varphi \rceil \supset^m \lceil \psi \rceil$
 $\mid \lceil \mathcal{B}^d i \varphi \rceil = \mathcal{B}^m i \lceil \varphi \rceil \mid \lceil \mathcal{D}^d i \varphi \rceil = \mathcal{D}^m i \lceil \varphi \rceil$

Faithfulness for the minimal embedding holds relative to the one fixed, full-domain model given by the meta-level constants.

theorem *Faithful2*:

$\forall w. \langle (\lambda x::w. True), B_0, D_0, V_0, U_0, E_0 \rangle, w \models^d \varphi \longleftrightarrow w \models^m \lceil \varphi \rceil$
 apply (*induct* φ) by *auto*

theorem *Faithful3*:

$\forall w. \langle (\lambda x::w. True), B_0, D_0, V_0, U_0, E_0 \rangle, w \models^s \langle \varphi \rangle \longleftrightarrow w \models^m \lceil \varphi \rceil$
 using *Faithful1a*[*of* φ] *Faithful2*[*of* φ] by *auto*

Minimal validity is sound for deep validity in the following precise sense: the minimally valid truth sets are exactly the images of the deeply valid formulas. (This ports lemma *Sound1* of *FaithfulPMLinHOL*, with a structured proof.)

theorem *SoundMin*: $(\models^m \psi) \longleftrightarrow (\exists \varphi. \psi = \lceil \varphi \rceil \wedge \models^d \varphi)$

proof

assume $L: \models^m \psi$

have $\psi = \lceil p^a \supset^d p^a \rceil$

proof (*rule ext*)

fix w show $\psi w = \lceil p^a \supset^d p^a \rceil w$ using L **unfolding** *ValM-def* by *simp*

qed

moreover have $\models^d (p^a \supset^d p^a)$ **unfolding** *ValD-def* by *simp*

ultimately show $\exists \varphi. \psi = \lceil \varphi \rceil \wedge \models^d \varphi$ by *blast*

next

assume $\exists \varphi. \psi = \lceil \varphi \rceil \wedge \models^d \varphi$

then obtain φ **where** $1: \psi = \lceil \varphi \rceil$ **and** $2: \models^d \varphi$ by *blast*

have $\forall w. \langle (\lambda x::w. True), B_0, D_0, V_0, U_0, E_0 \rangle, w \models^d \varphi$ using 2 **unfolding** *ValD-def* by *simp*

thus $\models^m \psi$ using 1 *Faithful2* **unfolding** *ValM-def* by *simp*

qed

A second use of the model finder: the two shallow backends are NOT interchangeable. Minimal validity of $\lceil \varphi \rceil$ does not entail deep validity of φ , since the fixed constants may satisfy a formula that fails in some other model.

nitpick refutes the implication; the witness below makes the refutation permanent: an atom true throughout the fixed model but false somewhere else.

lemma *MinimalNotDeep-countermodel*:

assumes *minimally-valid*: $\forall w. V_0 \ x \ w$ **and** *elsewhere-false*: $\neg V_1 \ x \ w_1$
shows $\models^m [x^a]$ **and** $\not\models^d x^a$

proof –

show $\models^m [x^a]$ **using** *minimally-valid* **unfolding** *ValM-def* **by** *simp*

next

show $\neg \models^d x^a$

proof

assume $A: \models^d x^a$

have $V_1 \ x \ w_1$

using $A[\text{unfolded } \text{ValD-def},$

$\text{THEN spec}[\text{where } x=\lambda w::w. \text{True}], \text{ THEN spec}[\text{where } x=B_0],$

$\text{THEN spec}[\text{where } x=D_0], \text{ THEN spec}[\text{where } x=V_1],$

$\text{THEN spec}[\text{where } x=U_0], \text{ THEN spec}[\text{where } x=E_0],$

$\text{THEN spec}[\text{where } x=w_1]]$ **by** *simp*

thus *False* **using** *elsewhere-false* **by** *simp*

qed

qed

Comparison. The maximal embedding threads the whole model through every formula: validity and the global consequence relation used by the protocol are genuinely model-quantified, faithfulness (Faithful1a/1b) is unrestricted, and bounded world domains remain expressible. The minimal embedding is leaner - one world argument instead of seven parameters - and matches the original HOMML/FATIO development; but its faithfulness (Faithful2) is relative to the one fixed full-domain model, so dialogue checking on this backend is checking IN a situation rather than a logically necessary entailment. Since the protocol layer is defined over the DEEP syntax, both backends serve it soundly; this entry uses the maximal one for the protocol's consequence relation and provides the minimal one, with the bridging theorems above, for lightweight automation and for continuity with the original sources.

end

6 The Fatio protocol over the deep syntax

theory *FatioFaithful-protocol*

imports *FatioFaithful-faithfulness*

begin

6.1 Dialectical obligation stores

datatype *DOSEntry* = *Entry Speaker Formula Sign*

type-synonym *DOS* = *DOSEntry list*

```

fun DOSUpdate :: FatioL  $\Rightarrow$  DOS  $\Rightarrow$  DOS where
  DOSUpdate assert[i, $\varphi$ ] dos = (Entry i  $\varphi \oplus$ ) # dos
  | DOSUpdate question[j,i, $\varphi$ ] dos = dos
  | DOSUpdate challenge[j,i, $\varphi$ ] dos = (Entry j  $\varphi \ominus$ ) # dos
  | DOSUpdate (Justify i  $\Phi$  sg  $\varphi$ ) dos = (Entry i  $\Phi$  sg) # dos
  | DOSUpdate retract[i, $\varphi$ ,sg] dos = removeAll (Entry i  $\varphi$  sg) dos

```

6.2 Pre-conditions (axiomatic semantics)

Two reconstruction decisions are made explicit here. (1) Following the sources of [7], justify is sign-parametric: *justify*[$i, \Phi \vdash^\ominus \varphi$] discharges a negative obligation incurred by a challenge. (2) The pre-condition of question is generalised so that the requested justification matches the SIGN of the questioned speaker's actual obligation; the Brigade Restaurant dialogue (where c questions the challenger b, whose obligation is negative) requires exactly this generalisation, which remains tacit in the original sources.

```

fun PreCond :: DOS  $\Rightarrow$  (BDF  $\Rightarrow$  bool)  $\Rightarrow$  FatioL  $\Rightarrow$  bool where
  PreCond dos  $\Gamma$  assert[i, $\varphi$ ] =
    ((Entry i  $\varphi \oplus$ )  $\notin$  set dos  $\wedge$  ( $\forall j. j \neq i \rightarrow \Gamma \models \mathcal{D}^d i (\mathcal{B}^d j (\mathcal{B}^d i \{\varphi\}^d)$ )))
  | PreCond dos  $\Gamma$  question[j,i, $\varphi$ ] =
    (i  $\neq$  j  $\wedge$  ( $\exists$  sg. (Entry i  $\varphi$  sg)  $\in$  set dos  $\wedge$ 
      ( $\forall k. k \neq j \rightarrow \Gamma \models \mathcal{D}^d j (\mathcal{B}^d k (\mathcal{D}^d j (ExJD i sg \varphi))$ ))))
  | PreCond dos  $\Gamma$  challenge[j,i, $\varphi$ ] =
    (i  $\neq$  j  $\wedge$  (Entry i  $\varphi \oplus$ )  $\in$  set dos  $\wedge$ 
      ( $\forall k. k \neq j \rightarrow \Gamma \models \mathcal{D}^d j (\mathcal{B}^d k (\neg^d (\mathcal{B}^d j \{\varphi\}^d))$ )  $\wedge$ 
      ( $\forall k. k \neq j \rightarrow \Gamma \models \mathcal{D}^d j (\mathcal{B}^d k (\mathcal{D}^d j (ExJD i \oplus \varphi))$ ))))
  | PreCond dos  $\Gamma$  (Justify i  $\Phi$  sg  $\varphi$ ) =
    ((Entry i  $\varphi$  sg)  $\in$  set dos  $\wedge$ 
      ( $\exists j. j \neq i \wedge (\Gamma (Done^d question[j,i,\varphi]) \vee \Gamma (Done^d challenge[j,i,\varphi]))$ )  $\wedge$ 
      ( $\forall k. k \neq i \rightarrow \Gamma \models \mathcal{D}^d i (\mathcal{B}^d k (\mathcal{B}^d i (EntD \Phi sg \varphi))$ ))))
  | PreCond dos  $\Gamma$  retract[i, $\varphi$ ,sg] =
    ((Entry i  $\varphi$  sg)  $\in$  set dos  $\wedge$ 
      (case sg of  $\oplus \Rightarrow (\forall j. j \neq i \rightarrow \Gamma \models \mathcal{D}^d i (\mathcal{B}^d j (\neg^d (\mathcal{B}^d i \{\varphi\}^d))$ )
        |  $\ominus \Rightarrow (\forall j. j \neq i \rightarrow \Gamma \models \mathcal{D}^d i (\mathcal{B}^d j (\neg^d \neg^d (\mathcal{B}^d i \{\varphi\}^d))$ ))))

```

6.3 Post-conditions and state evolution

```

fun GammaAdd :: FatioL  $\Rightarrow$  (BDF  $\Rightarrow$  bool) where
  GammaAdd assert[i, $\varphi$ ] =
    ( $\lambda x. \exists k j. k \neq i \wedge j \neq i \wedge x = \mathcal{B}^d k (\mathcal{D}^d i (\mathcal{B}^d j (\mathcal{B}^d i \{\varphi\}^d))$ ))
  | GammaAdd question[j,i, $\varphi$ ] = ( $\lambda x. False$ )
  | GammaAdd challenge[j,i, $\varphi$ ] = ( $\lambda x. False$ )
  | GammaAdd (Justify i  $\Phi$  sg  $\varphi$ ) =
    ( $\lambda x. \exists k j. k \neq i \wedge j \neq i \wedge x = \mathcal{B}^d k (\mathcal{D}^d i (\mathcal{B}^d j (\mathcal{B}^d i (EntD \Phi sg \varphi))$ ))
  | GammaAdd retract[i, $\varphi$ ,sg] =
    (case sg of  $\oplus \Rightarrow (\lambda x. \exists k j. k \neq i \wedge j \neq i \wedge x = \mathcal{B}^d k (\mathcal{D}^d i (\mathcal{B}^d j (\neg^d (\mathcal{B}^d i \{\varphi\}^d))$ ))
      |  $\ominus \Rightarrow (\lambda x. \exists k j. k \neq i \wedge j \neq i \wedge x = \mathcal{B}^d k (\mathcal{D}^d i (\mathcal{B}^d j (\neg^d \neg^d (\mathcal{B}^d i \{\varphi\}^d))$ ))

```

$\{\varphi\}^d))))))$

definition $\text{GammaUpdate} :: \text{FatioL} \Rightarrow (\text{BDF} \Rightarrow \text{bool}) \Rightarrow (\text{BDF} \Rightarrow \text{bool})$ **where**
 $\text{GammaUpdate } l \ \Gamma \equiv \lambda x. \text{GammaAdd } l \ x \vee \Gamma \ x \vee x = \text{Done}^d \ l$

fun $\text{FatioCheckRec} :: \text{FatioL} \ \text{list} \Rightarrow \text{DOS} \Rightarrow (\text{BDF} \Rightarrow \text{bool}) \Rightarrow \text{bool}$ **where**
 $\text{FatioCheckRec } [] \ \text{dos } \Gamma = \text{True}$
 $| \text{FatioCheckRec } (l \# ls) \ \text{dos } \Gamma =$
 $(\text{PreCond } \text{dos } \Gamma \ l \wedge \text{FatioCheckRec } ls \ (\text{DOSUpdate } l \ \text{dos}) \ (\text{GammaUpdate } l$
 $\Gamma))$

6.4 Sanity theorems

lemma $\text{MemberEntails[intro]}: \Gamma \ \varphi \Longrightarrow \Gamma \models \varphi$ **unfolding** ConsD-def **by** blast

lemma $\text{GammaMono}: \Gamma \ x \Longrightarrow \text{GammaUpdate } l \ \Gamma \ x$ **by** $(\text{simp add: GammaUp-}$
 $\text{date-def})$

lemma $\text{DoneRecorded}: \text{GammaUpdate } l \ \Gamma \ (\text{Done}^d \ l)$ **by** $(\text{simp add: GammaUp-}$
 $\text{date-def})$

lemma $\text{AssertCreatesObligation}: (\text{Entry } i \ \varphi \oplus) \in \text{set } (\text{DOSUpdate } \text{assert}[i, \varphi] \ \text{dos})$
by simp

lemma $\text{RetractRemovesObligation}: (\text{Entry } i \ \varphi \ \text{sg}) \notin \text{set } (\text{DOSUpdate } \text{retract}[i, \varphi, \text{sg}]$
 $\text{dos})$ **by** simp

end

7 Tests: two dialogues, checked with high automa- tion

theory $\text{FatioFaithful-tests}$

imports $\text{FatioFaithful-protocol}$

begin

7.1 The full Brigade Restaurant dialogue (from the litera- ture)

The six-locution dialogue of [7], going back to [6]: a asserts that the Brigade restaurant is good (r); b challenges; a justifies from the newspaper review (n); c questions the challenger b, whose obligation is NEGATIVE; b justifies negatively from the chef change and quality drop ($m \wedge^c s$); finally a retracts. The initial knowledge Γ_B lists exactly the mental-state formulas required by the pre-conditions.

definition $\Gamma_B :: \text{BDF} \Rightarrow \text{bool}$ **where** $\Gamma_B \equiv \lambda x.$

$(\exists j. j \neq a \wedge x = \mathcal{D}^d \ a \ (\mathcal{B}^d \ j \ (\mathcal{B}^d \ a \ \{\mathcal{r}^c\}^d)))$
 $\vee (\exists k. k \neq b \wedge x = \mathcal{D}^d \ b \ (\mathcal{B}^d \ k \ (\neg^d (\mathcal{B}^d \ b \ \{\mathcal{r}^c\}^d))))$
 $\vee (\exists k. k \neq b \wedge x = \mathcal{D}^d \ b \ (\mathcal{B}^d \ k \ (\mathcal{D}^d \ b \ (\text{ExJD } a \oplus (\mathcal{r}^c))))))$
 $\vee (\exists k. k \neq a \wedge x = \mathcal{D}^d \ a \ (\mathcal{B}^d \ k \ (\mathcal{B}^d \ a \ (\text{EntD } (n^c) \oplus (\mathcal{r}^c))))))$
 $\vee (\exists k. k \neq c \wedge x = \mathcal{D}^d \ c \ (\mathcal{B}^d \ k \ (\mathcal{D}^d \ c \ (\text{ExJD } b \ominus (\mathcal{r}^c))))))$

$$\begin{aligned} & \vee (\exists k. k \neq b \wedge x = \mathcal{D}^d b (\mathcal{B}^d k (\mathcal{B}^d b (\text{EntD} (m^c \wedge^c s^c) \ominus (r^c)))))) \\ & \vee (\exists j. j \neq a \wedge x = \mathcal{D}^d a (\mathcal{B}^d j (\neg^d (\mathcal{B}^d a \{r^c\}^d)))) \end{aligned}$$

abbreviation *Brigade* $\equiv$

$$\begin{aligned} & [\text{assert}[a, r^c], \text{challenge}[b, a, r^c], \text{justify}[a, n^c \vdash^\oplus r^c], \\ & \text{question}[c, b, r^c], \text{justify}[b, m^c \wedge^c s^c \vdash^\ominus r^c], \text{retract}[a, r^c, \oplus]] \end{aligned}$$

Automation. Every step obligation is discharged by a single automated call (*auto* with the two protocol definitions and *MemberEntails* as an introduction rule). The justify steps additionally exhibit their Done-witness in two lines beforehand, and the question steps select the sign of the addressed obligation. This division of labour is deliberate: *sledgehammer* [4] finds no proof for these steps even with generous time limits and the relevant facts supplied, because what they require is the choice of a WITNESS over a finite datatype (which speaker questioned or challenged, which sign the obligation carries) rather than a derivation; automated provers are strong at the latter and weak at the former. Supplying the witness explicitly and automating the rest is therefore not a workaround but the appropriate split.

abbreviation $\text{dos}_1 \equiv \text{DOSUpdate } \text{assert}[a, r^c] []$

abbreviation $\text{dos}_2 \equiv \text{DOSUpdate } \text{challenge}[b, a, r^c] \text{ dos}_1$

abbreviation $\text{dos}_3 \equiv \text{DOSUpdate } \text{justify}[a, n^c \vdash^\oplus r^c] \text{ dos}_2$

abbreviation $\text{dos}_4 \equiv \text{DOSUpdate } \text{question}[c, b, r^c] \text{ dos}_3$

abbreviation $\text{dos}_5 \equiv \text{DOSUpdate } \text{justify}[b, m^c \wedge^c s^c \vdash^\ominus r^c] \text{ dos}_4$

abbreviation $\Gamma_1 \equiv \text{GammaUpdate } \text{assert}[a, r^c] \Gamma_B$

abbreviation $\Gamma_2 \equiv \text{GammaUpdate } \text{challenge}[b, a, r^c] \Gamma_1$

abbreviation $\Gamma_3 \equiv \text{GammaUpdate } \text{justify}[a, n^c \vdash^\oplus r^c] \Gamma_2$

abbreviation $\Gamma_4 \equiv \text{GammaUpdate } \text{question}[c, b, r^c] \Gamma_3$

abbreviation $\Gamma_5 \equiv \text{GammaUpdate } \text{justify}[b, m^c \wedge^c s^c \vdash^\ominus r^c] \Gamma_4$

lemma *B1*: $\text{PreCond} [] \Gamma_B \text{ assert}[a, r^c]$

by (*auto simp*: $\Gamma_B\text{-def intro!}$: *MemberEntails*)

lemma *B2*: $\text{PreCond } \text{dos}_1 \Gamma_1 \text{ challenge}[b, a, r^c]$

by (*auto simp*: $\Gamma_B\text{-def GammaUpdate-def intro!}$: *MemberEntails*)

lemma *B3*: $\text{PreCond } \text{dos}_2 \Gamma_2 \text{ justify}[a, n^c \vdash^\oplus r^c]$

proof –

have $\Gamma_2 (\text{Done}^d \text{ challenge}[b, a, r^c])$ **by** (*simp add*: *GammaUpdate-def*)

hence $\exists j. j \neq a \wedge (\Gamma_2 (\text{Done}^d \text{ question}[j, a, r^c]) \vee \Gamma_2 (\text{Done}^d \text{ challenge}[j, a, r^c]))$

by (*intro exI*[*of* - *b*]) *simp*

thus *?thesis* **by** (*auto simp*: $\Gamma_B\text{-def GammaUpdate-def intro!}$: *MemberEntails*)

qed

lemma *B4*: $\text{PreCond } \text{dos}_3 \Gamma_3 \text{ question}[c, b, r^c]$

by (*intro PreCond.simps*[*THEN iffD2*] *conjI exI*[*of* - $\ominus$])

(*auto simp*: $\Gamma_B\text{-def GammaUpdate-def intro!}$: *MemberEntails*)

lemma *B5*: $\text{PreCond } \text{dos}_4 \Gamma_4 \text{ justify}[b, m^c \wedge^c s^c \vdash^\ominus r^c]$

proof –

have $\Gamma_4 (\text{Done}^d \text{ question}[c, b, r^c])$ **by** (*simp add*: *GammaUpdate-def*)

hence $\exists j. j \neq b \wedge (\Gamma_4 (\text{Done}^d \text{ question}[j, b, r^c]) \vee \Gamma_4 (\text{Done}^d \text{ challenge}[j, b, r^c]))$

by (*intro exI*[*of* - *c*]) *simp*

thus ?thesis by (auto simp: Γ_B -def GammaUpdate-def intro!: MemberEntails)
 qed
 lemma B6: PreCond dos₅ Γ_5 retract[a,r^c,⊕]
 by (auto simp: Γ_B -def GammaUpdate-def intro!: MemberEntails)
 theorem BrigadeDialogue: FatioCheckRec Brigade [] Γ_B
 using B1 B2 B3 B4 B5 B6 by simp

7.2 A created example: role reversal with a negative retraction

Seven locutions over two topics: after the exchange on p is settled by b RETRACTING ITS NEGATIVE obligation (exercising the $\ominus$ -clause of retract), the roles reverse and b becomes theasserter of q, questioned by a.

definition $\Gamma_C :: BDF \Rightarrow bool$ where $\Gamma_C \equiv \lambda x.$

$(\exists j. j \neq a \wedge x = \mathcal{D}^d a (\mathcal{B}^d j (\mathcal{B}^d a \{p^c\}^d)))$
 $\vee (\exists k. k \neq b \wedge x = \mathcal{D}^d b (\mathcal{B}^d k (\neg^d (\mathcal{B}^d b \{p^c\}^d))))$
 $\vee (\exists k. k \neq b \wedge x = \mathcal{D}^d b (\mathcal{B}^d k (\mathcal{D}^d b (ExJD a \oplus (p^c)))))$
 $\vee (\exists k. k \neq a \wedge x = \mathcal{D}^d a (\mathcal{B}^d k (\mathcal{B}^d a (EntD (n^c) \oplus (p^c)))))$
 $\vee (\exists j. j \neq b \wedge x = \mathcal{D}^d b (\mathcal{B}^d j (\neg^d \neg^d (\mathcal{B}^d b \{p^c\}^d))))$
 $\vee (\exists j. j \neq b \wedge x = \mathcal{D}^d b (\mathcal{B}^d j (\mathcal{B}^d b \{q^c\}^d)))$
 $\vee (\exists k. k \neq a \wedge x = \mathcal{D}^d a (\mathcal{B}^d k (\mathcal{D}^d a (ExJD b \oplus (q^c)))))$
 $\vee (\exists k. k \neq b \wedge x = \mathcal{D}^d b (\mathcal{B}^d k (\mathcal{B}^d b (EntD (m^c) \oplus (q^c)))))$

abbreviation Cascade $\equiv$

$[\text{assert}[a,p^c], \text{challenge}[b,a,p^c], \text{justify}[a,n^c \vdash^\oplus p^c], \text{retract}[b,p^c,\ominus],$
 $\text{assert}[b,q^c], \text{question}[a,b,q^c], \text{justify}[b,m^c \vdash^\oplus q^c]]$

abbreviation cd₁ $\equiv$ DOSUpdate assert[a,p^c] []

abbreviation cd₂ $\equiv$ DOSUpdate challenge[b,a,p^c] cd₁

abbreviation cd₃ $\equiv$ DOSUpdate justify[a,n^c⊢[⊕]p^c] cd₂

abbreviation cd₄ $\equiv$ DOSUpdate retract[b,p^c,⊖] cd₃

abbreviation cd₅ $\equiv$ DOSUpdate assert[b,q^c] cd₄

abbreviation cd₆ $\equiv$ DOSUpdate question[a,b,q^c] cd₅

abbreviation Δ₁ $\equiv$ GammaUpdate assert[a,p^c] Γ_C

abbreviation Δ₂ $\equiv$ GammaUpdate challenge[b,a,p^c] Δ₁

abbreviation Δ₃ $\equiv$ GammaUpdate justify[a,n^c⊢[⊕]p^c] Δ₂

abbreviation Δ₄ $\equiv$ GammaUpdate retract[b,p^c,⊖] Δ₃

abbreviation Δ₅ $\equiv$ GammaUpdate assert[b,q^c] Δ₄

abbreviation Δ₆ $\equiv$ GammaUpdate question[a,b,q^c] Δ₅

lemma C1: PreCond [] Γ_C assert[a,p^c]

by (auto simp: Γ_C -def intro!: MemberEntails)

lemma C2: PreCond cd₁ Δ₁ challenge[b,a,p^c]

by (auto simp: Γ_C -def GammaUpdate-def intro!: MemberEntails)

lemma C3: PreCond cd₂ Δ₂ justify[a,n^c⊢[⊕]p^c]

proof –

have Δ₂ (Done^d challenge[b,a,p^c]) by (simp add: GammaUpdate-def)

hence $\exists j. j \neq a \wedge (\Delta_2 (\text{Done}^d \text{question}[j,a,p^c]) \vee \Delta_2 (\text{Done}^d \text{challenge}[j,a,p^c]))$

```

    by (intro exI[of - b]) simp
  thus ?thesis by (auto simp:  $\Gamma_C$ -def GammaUpdate-def intro!: MemberEntails)
qed
lemma C4: PreCond cd3  $\Delta_3$  retract[b, pc,  $\ominus$ ]
  by (auto simp:  $\Gamma_C$ -def GammaUpdate-def intro!: MemberEntails)
lemma C5: PreCond cd4  $\Delta_4$  assert[b, qc]
  by (auto simp:  $\Gamma_C$ -def GammaUpdate-def intro!: MemberEntails)
lemma C6: PreCond cd5  $\Delta_5$  question[a, b, qc]
  by (intro PreCond.simps[THEN iffD2] conjI exI[of -  $\oplus$ ])
  (auto simp:  $\Gamma_C$ -def GammaUpdate-def intro!: MemberEntails)
lemma C7: PreCond cd6  $\Delta_6$  justify[b, mc  $\vdash^{\oplus}$  qc]
proof -
  have  $\Delta_6$  (Doned question[a, b, qc]) by (simp add: GammaUpdate-def)
  hence  $\exists j. j \neq b \wedge (\Delta_6$  (Doned question[j, b, qc])  $\vee \Delta_6$  (Doned challenge[j, b, qc]))
    by (intro exI[of - a]) simp
  thus ?thesis by (auto simp:  $\Gamma_C$ -def GammaUpdate-def intro!: MemberEntails)
qed

theorem CascadeDialogue: FatioCheckRec Cascade []  $\Gamma_C$ 
  using C1 C2 C3 C4 C5 C6 C7 by simp

end

```

References

- [1] C. Benzmüller. Faithful logic embeddings in HOL – deep and shallow. *CoRR*, abs/2607.10880, 2025.
- [2] C. Benzmüller. Faithful propositional modal logic embeddings in HOL. *Archive of Formal Proofs*, 2025. <https://isa-afp.org/entries/FaithfulPMLinHOL.html>, Formal proof development.
- [3] C. Benzmüller, X. Parent, and L. van der Torre. Designing normative theories for ethical and legal reasoning: LogiKEy framework, methodology, and tool support. *Artificial Intelligence*, 287:103348, 2020.
- [4] J. C. Blanchette, S. Böhme, and L. C. Paulson. Extending sledgehammer with SMT solvers. In *CADE-23*, volume 6803 of *LNCS*, pages 116–130. Springer, 2011.
- [5] J. C. Blanchette and T. Nipkow. *Nitpick: A Counterexample Generator for Higher-Order Logic Based on a Relational Model Finder*. Springer, ITP 2010, LNCS 6172, pages 131–146, 2010.
- [6] P. McBurney and S. Parsons. Locutions for argumentation in agent interaction protocols. In *Agent Communication*, volume 3396 of *LNCS*, pages 209–225. Springer, 2005.

- [7] L. Pasetto and C. Benzmüller. Implementing the Fatio protocol for multi-agent argumentation in LogiKEy. In *Proceedings of the 4th International Workshop on Automated Reasoning in Quantified Non-Classical Logics (ARQNL 2024)*, volume 3875 of *CEUR Workshop Proceedings*. CEUR-WS.org, 2024.